pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 6: ejercicio

Otras estructuras de datos - Ejercicio

Daniel Trabas

pontIA

Índice

Ejercicio práctico avanzado: Gestión de tripulación y manifiestos de vuelo 3

Objetivo 4

Justificación requerida 5

La entrega consistirá en un único archivo comprimido (.zip) que contenga: 5

PontIA

Ejercicio práctico avanzado: Gestión de tripulación y manifiestos de vuelo

La aerolínea global integrada en nuestro esquema de base de datos bookings (demodb) requiere expandir sus operaciones lógicas de auditoría y control de vuelos. Hasta ahora, el sistema gestiona los vuelos, pasajes y asignación de asientos de los pasajeros de forma impecable bajo garantías ACID.

Sin embargo, se ha detectado un vacío de información crítico en el control: la base de datos no contiene registro de la tripulación de vuelo asignada a cada trayecto ni archiva los manifiestos de pasajeros registrados que exigen las autoridades de aviación civil al cierre de cada puerta de embarque.

Tu tarea es diseñar y desplegar de forma segura el nuevo subsistema de tripulaciones y auditoría física de manifiestos de vuelo en PostgreSQL, implementando estructuras avanzadas que optimicen la memoria y la velocidad del motor relacional.

Requisitos del Ejercicio

Se debe desarrollar un script SQL (.sql) estructurado y auto-ejecutable que resuelva de forma exacta las siguientes fases de implementación en el esquema bookings de la base de datos demodb:

Fase A: Creación de tipo enumerado personalizado

Declara un tipo de datos propio de tipo enumerado (ENUM) llamado bookings.crew_role_type que restrinja de forma física los cargos permitidos dentro de las aeronaves. Los valores permitidos son exclusivamente:

- 'Captain' (Comandante)
- 'First Officer' (Copiloto)
- 'Purser' (Sobrecargo o Jefe de Cabina)
- 'Cabin Crew' (Tripulante de Cabina de Pasajeros / Auxiliar)

Fase B: Creación de la tabla de tripulantes (DDL)

Crea la tabla física bookings.flight_crew para registrar las hojas de vida de los tripulantes. Debe cumplir con:

- **crew_id:** clave primaria de tipo texto (varchar/text) que almacena el identificador con el formato 'XX 99999999X', validado por un CHECK constraint de expresión regular.
- **crew_name:** nombre completo del tripulante (text, no nulo).
- **role:** el cargo asignado utilizando el tipo ENUM bookings.crew_role_type creado en la Fase A (no nulo).
- **hire_date:** fecha de contratación de tipo date (por defecto la fecha actual del sistema).

Fase C: Tabla de manifiestos de pasajeros (gestión BLOB)

Diseña la tabla bookings.boarding_manifests que actuará como archivo digital de los manifiestos de pasajeros firmados en PDF. La tabla debe relacionarse directamente con los vuelos existentes para evitar inconsistencias de datos:

- **manifest_id:** serial PRIMARY KEY.
- **flight_id:** clave foránea que referencia a bookings.flights(flight_id) con reglas activas ON UPDATE CASCADE ON DELETE CASCADE.
- **pdf_manifest:** campo para almacenar el archivo binario del PDF. El alumno debe justificar técnicamente en su informe si utilizará el tipo directo BYTEA o el subsistema de punteros OID de pg_largeobject.

Fase D: Clonación y población del historial con SELECT INTO

Para iniciar la tabla de asignación de turnos de tripulación, utiliza la instrucción SELECT INTO para generar una nueva tabla física llamada bookings.flight_crew_assignments clonando dinámicamente la estructura de los vuelos que ya tengan un estado completado ('Arrived' o 'Departed') desde la tabla bookings.flights. Durante esta instrucción de creación ágil, debes:

- Seleccionar las columnas flight_id, route_no, scheduled_departure y status.
- Inyectar un campo vacío (NULL) de tipo texto para el crew_id que actuará como marcador de posición para la asignación de tripulación.
- Añadir la restricción de clave primaria (PRIMARY KEY) compuesta sobre (flight_id, crew_id) de forma posterior utilizando un ALTER TABLE.
- Insertar al menos 3 registros consistentes asociando tripulantes reales creados en la Fase B con vuelos de esta nueva tabla.

Fase E: Generación de vista analitica (vw_flight_crew_roster)

Genera la vista relacional bookings.vw_flight_crew_roster para que el departamento de operaciones consuma el itinerario de vuelos junto con los nombres de la tripulación de forma simplificada. La vista debe unir de forma horizontal las tablas de asignaciones, vuelos, rutas y la tabla de tripulantes.

Fase E: Generación de vista analitica (vw_flight_crew_roster)

Desarrolla un procedimiento almacenado llamado bookings.get_flight_crew_details que reciba dos parámetros de entrada:

- p_flight_id (integer)
- Cargo deseado p_role (de tipo ENUM crew_role_type)

El procedimiento debe retornar (mediante parámetros INOUT o un cursor de salida) un registro detallado que contenga:

- El nombre del tripulante (extraído de la tabla de tripulación).
- El cargo textual (procedente del tipo de datos propio ENUM).
- El estado del vuelo actual.
- Un dato específico extraído de un campo JSON:
  - Nombre del piloto asignado
  - La velocidad de viento registrada de forma dinámica desde la tabla de metadatos de vuelo (bookings.flight_metadata) que creamos en lecciones anteriores.

Asegúrate de aplicar control transaccional e introspección de errores en PL/pgSQL.

Objetivo

El objetivo primordial es que demuestres tu dominio en la creación y manipulación de estructuras relacionales en PostgreSQL (operaciones DDL y DML), manejo de diferentes tipos de datos y encapsulado de consultas.

Justificación requerida

No basta con entregar un código sintácticamente correcto. Debes justificar el porqué de las decisiones de diseño adoptadas en este ejercicio frente a esquemas desnormalizados o de baja integridad. Para ello te propongo utilizar una tabla como la siguiente:

Elemento de diseño	Propuesta técnica	Justificación relacional e impacto en rendimiento
‹Elemento, Tabla, vista, etc.›	‹Tipo de configuración, de dato, de ejecución, etc.›	‹Detalle de la justificación, el porqué lo hemos hecho así›

La entrega consistirá en un único archivo comprimido (.zip) que contenga:

1. El script SQL estructurado y limpio conteniendo todas las fases lógicas.
2. Un documento de texto o PDF de justificación técnica respondiendo a las preguntas de arquitectura (Fase 5).

Puntuación Máxima: 10 Puntos

- **2.0 Puntos:** correcta definición y CHECK de ID del tripulante en la Fase B (ENUM + regex).
- **2.0 Puntos:** justificación y diseño de almacenamiento del manifiesto PDF en la Fase C.
- **2.0 Puntos:** implementación de SELECT INTO y clave primaria en la Fase D.
- **2.0 Puntos:** creación de la vista relacional de forma correcta.
- **2.0 Puntos:** diseño y ejecución exitosa del procedimiento almacenado con extracción JSONB y ENUM.

PONTIA